Personalized Networking Assistant

Project Description:

The Personalized Networking Assistant (PNA) is an AI-powered application designed to help users build professional connections by generating personalized conversation starters, networking topics, and verified information based on their interests and event context. The system improves networking confidence by providing intelligent suggestions that make professional interactions more engaging and meaningful.

The application is built using Python, FastAPI, Streamlit, DistilBERT, GPT-2 Small, and the Wikipedia API. DistilBERT is used for zero-shot topic classification, GPT-2 Small generates personalized conversation starters, and the Wikipedia API provides fact-checking and contextual information. FastAPI manages backend APIs, while Streamlit offers an interactive and user-friendly interface.

The application provides features such as personalized profile creation, AI-generated conversation starters, topic recommendations, Wikipedia-based fact verification, conversation history tracking, and feedback collection. These features help users prepare for networking events, interviews, conferences, and professional meetings.

The project serves students, job seekers, professionals, entrepreneurs, and researchers by improving communication confidence, networking skills, and professional relationship building.

Scenarios

Scenario 1: Personalized Conversation Generation

A user enters their profession, interests, and networking event details. The system uses GPT-2 Small to generate personalized conversation starters that match the user's background and the event context, helping initiate meaningful conversations.

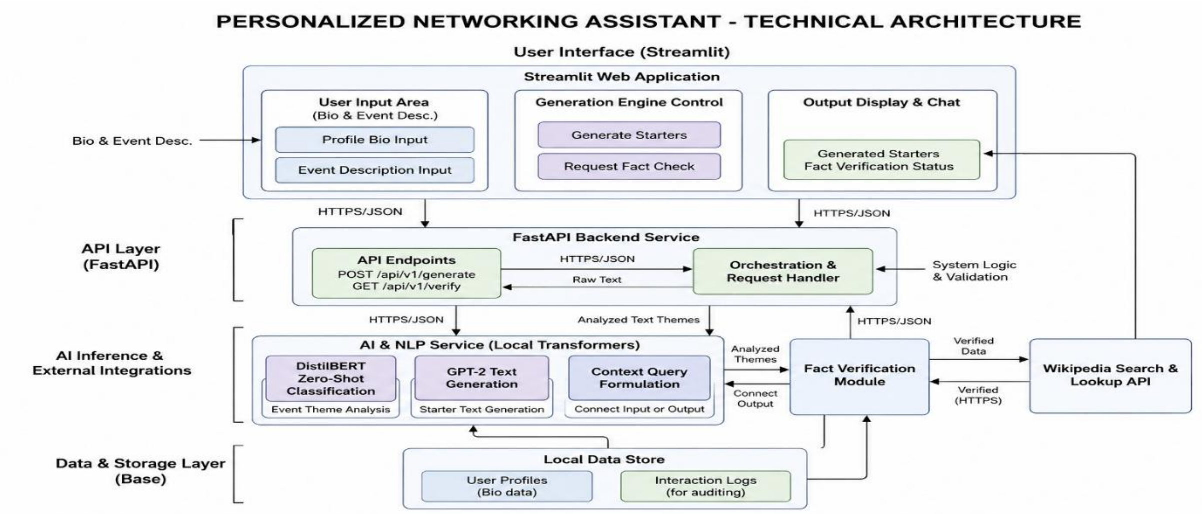
Scenario 2: Intelligent Topic Recommendation and Fact Verification

When the user selects a discussion topic, DistilBERT classifies the topic into relevant categories while the Wikipedia API retrieves verified information. This helps users confidently discuss unfamiliar subjects with accurate facts.

Scenario 3: Networking History and Feedback Analysis

After each interaction, the system stores generated conversation starters and user feedback. Users can review previous conversations, monitor their networking progress, and continuously improve their communication skills.

Technical Architecture:



Description: The Personalized Networking Assistant follows a modular AI-driven architecture that integrates user profile analysis, AI-powered conversation generation, topic classification, fact verification, and feedback management into a single networking assistance platform.

The Streamlit frontend allows users to enter their profile, interests, and event details while viewing AI-generated conversation starters, suggested topics, verified facts, conversation history, and feedback. The FastAPI backend manages API requests and communicates with GPT-2 Small for conversation generation, DistilBERT for topic classification, and the Wikipedia API for fact verification. User history and feedback are stored locally for future reference. The modular architecture ensures scalability, maintainability, and efficient communication between the frontend, backend, AI models, and external APIs.

Pre-requisites:

Python 3.10+: https://docs.python.org/3/

Streamlit:https: //docs.streamlit.io/

FastAPI:https: //fastapi.tiangolo.com/

Uvicorn:https: //www.uvicorn.org/

Transformers (Hugging Face): https://huggingface.co/docs/transformers/

PyTorch (Torch): https://pytorch.org/docs/stable/

Wikipedia API (wikipedia library): https://pypi.org/project/wikipedia/

NumPy:https: //numpy.org/doc/

Pandas:https: //pandas.pydata.org/docs/

Requests:https: //requests.readthedocs.io/

Git:https: //git-scm.com/doc

GitHub:https: //docs.github.com/

Visual Studio Code:https: //code.visualstudio.com/docs

Project Workflow:

Milestone 1: Milestone 1: Project Planning & Architecture

- **Activity 1.1:** Milestone 1: Project Planning & Architecture
- **Activity 1.2:** Research AI models (GPT-2 Small, DistilBERT) and supporting APIs.
- **Activity 1.3:** Design the system architecture and workflow.
- **Activity 1.4:** Set up the Python development environment and required libraries.

Milestone 2: Backend & AI Development

- **Activity 2.1:** Develop the FastAPI backend services
- **Activity 2.2:** Integrate GPT-2 Small for conversation generation
- **Activity 2.3:** Integrate DistilBERT for topic classification
- **Activity 2.4:** * Integrate Wikipedia API for fact verification

Milestone 3: Frontend Development

- **Activity 3.1:** Build the Streamlit web application
- **Activity 3.2:** Create profile and event input forms
- **Activity 3.3:** Display conversation starters, verified facts, and AI suggestions
- **Activity 3.3:** Implement conversation history and feedback pages

Milestone 4: Data Management & Integration

- **Activity 4.1:** Store user profiles and interaction history
- **Activity 4.2:** Save feedback and conversation logs.
- **Activity 4.3:** Connect frontend with backend APIs.
- **Activity 4.4:** Validate AI responses and optimize data flow.

Milestone 5: Testing & Deployment

- **Activity 5.1:** Perform functional and integration testing.
- **Activity 5.2:** Fix bugs and optimize application performance.
- **Activity 5.3:** Deploy the application locally and prepare project documentation.

Milestone 6: Conclusion

- Demonstrate the complete Personalized Networking Assistant, evaluate results, and document future enhancements.

Milestone 1: Project Planning & Architecture

In this milestone, we focused on understanding the networking challenges faced by professionals and designing an AI-powered solution. The system architecture, AI models, backend services, and development environment were finalized before implementation.

Activity 1.1: Gather Project Requirements

Before development, the project requirements were gathered and analyzed to understand the needs of professionals attending networking events.

- **Step 1 — Identify User Requirements:** Identify User Requirements: Define user profile, event details, networking goals, and interests.

Problem – Solution Fit Canvas

Customer Segments	Problems	Existing Alternatives	Desired Outcomes / Goals
• Professionals • Job Seekers • Entrepreneurs • Students • Event Attendees • Networkers	• Difficulty starting meaningful conversations with new people • Uncertainty about relevant topics to discuss • Hard to keep conversations engaging • Lack of verified information to share during discussions • No central place to save conversation history and feedback	• General ice-breaker websites • Random conversation suggestion apps • Manual search on the internet • Social media groups • Traditional networking guides	• Get personalized conversation starters • Receive relevant topic suggestions • Verify facts instantly • Maintain conversation history • Improve networking confidence and effectiveness

Proposed Solution	Key Benefits
The Personalized Networking Assistant is an AI-powered application that generates personalized conversation starters, classifies relevant topics, verifies facts using Wikipedia API, and maintains conversation history and feedback to help users build meaningful and impactful professional connections.	• AI-generated conversation starters tailored to user profile and event context • Relevant topic classification for effective discussions • Fact verification using trusted sources (Wikipedia) • Save and review past conversations and feedback • Enhances confidence and improves networking outcomes

- **Step 2 —Analyze Networking Challenges:** Analyze Networking Challenges: Study problems such as starting conversations, selecting relevant topics, and verifying shared information.

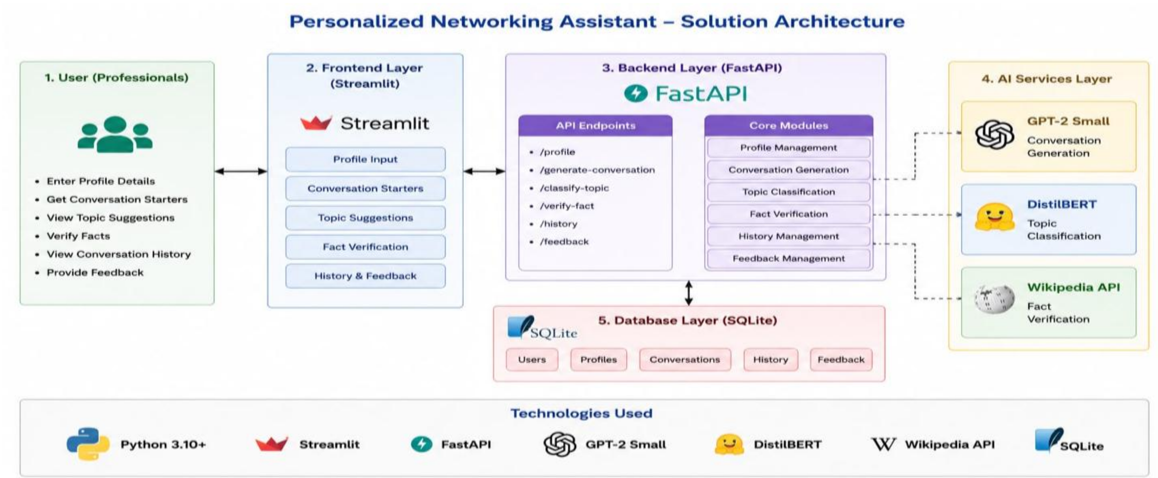
AI Models and Technologies Research

Technology / Model	Purpose	Why Selected	Key Features	Source / Documentation
GPT-2 Small OpenAI	Conversation Generation	Generates context-aware and natural conversation starters for networking scenarios.	• Pre-trained language model • Good for text generation • Lightweight and efficient • Easy integration	Hugging Face https://huggingface.co/gpt2
DistilBERT Hugging Face	Topic Classification	Efficient and accurate model for classifying networking topics.	• Distilled version of BERT • Fast and accurate • Works well for text classification • Low computational cost	Hugging Face https://huggingface.co/distilbert-base-uncased
Wikipedia API WIKIPEDIA The Free Encyclopedia	Fact Verification	Provides reliable and up-to-date information to verify facts shared in conversations.	• Free and open source • Large knowledge base • Easy API integration • Real-time information retrieval	WIKIPEDIA https://www.mediawiki.org/wiki/API:Main_page

Key Models Selected for Implementation

GPT-2 Small	DistilBERT	Wikipedia API
For AI-powered conversation generation and suggestions.	For networking topic classification and category prediction.	For fact verification and trusted information retrieval.

- **Step 3 —Finalize Functional Requirements:** Finalize features including AI conversation generation, topic classification, fact verification, and conversation history.



Activity 1.2: Research and Select the Appropriate Generative AI Model

The following AI technologies were researched and selected based on their performance and suitability for professional networking.

- **Understand Project Requirements:** Analyze the need for AI-generated networking conversations.
- **Explore AI Documentation:** Review GPT-2 Small, DistilBERT, Hugging Face Transformers, and Wikipedia API.
- **Evaluate Model Performance:** Compare response quality, speed, and resource utilization.
- **Select Optimal Models:** Finalize GPT-2 Small for conversation generation, DistilBERT for topic classification, and Wikipedia API for fact verification. Activity 1.3: Define the Architecture of the Application

Activity 1.3: Design the system architecture and workflow.

Design the overall architecture consisting of the Streamlit frontend, FastAPI backend, AI modules, and SQLite database.

- **Define Frontend Workflow:** Design pages for profile creation, conversation generation, and results display.
- **Develop Backend Modules:** Implement API endpoints and AI processing modules.
- **Integrate AI Components:** Connect GPT-2 Small, DistilBERT, Wikipedia API, and SQLite through FastAPI.

Activity 1.4: Set up the Python development environment and required libraries.

Set up the Python environment and project structure required to run the Voice Based Concept Understanding Analyser application.

- **Install Python and Pip:** Ensure Python 3.10+ is installed along with pip for package management.
- **Create a Virtual Environment:** Create a separate virtual environment to manage project dependencies safely.

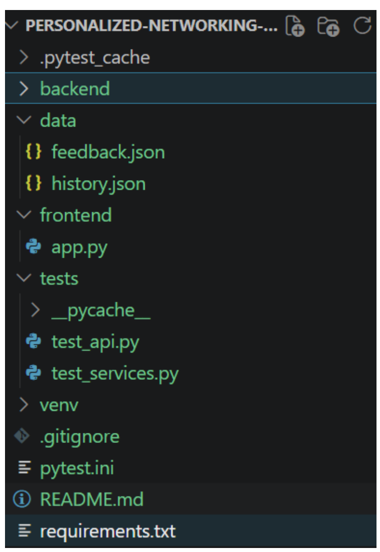
```
python -m venv venv
```

- **Activate the Virtual Environment:**

```
venv\Scripts\activate
```

```
pip install -r requirements.txt
```

- **Set Up Application Structure**: Organize the project files into separate modules for transcription, semantic analysis, audio processing, scoring, and report generation.



Milestone 2: Backend & AI Development

Activity 2.1 : Develop the FastAPI Backend Services:

Step 1 – Create FastAPI Application: Set up the backend application using FastAPI.

Step 2 – Define API Routes: Create endpoints for conversation generation, topic classification, fact verification, history, and feedback.

Step 3 – Validate Requests: Use Pydantic models to validate user inputs.

Step 4 – Connect Service Modules: Link backend routes with AI and utility service files.

Activity 2.2: Integrate GPT-2 Small for Conversation Generation:

Step 1 – Load GPT-2 Small Model: Use Hugging Face Transformers to load the GPT-2 Small text generation model.

Step 2 – Prepare Prompt: Combine user interests and event details into a structured prompt.

Step 3 – Generate Conversation Starters: Produce personalized networking conversation starters.

Step 4 – Return Results: Send generated suggestions back to the frontend through FastAPI.

Activity 2.3: Integrate DistilBERT for Topic Classification Extract waveform using Librosa.

Step 1 – Load DistilBERT Model: Use Hugging Face Transformers to load the DistilBERT classification model.

Step 2 – Analyze Event Description: Process the event details entered by the user.

Step 3 – Classify Relevant Topics: Identify suitable networking topics based on the user's input.

Step 4 – Return Classified Topics: Display extracted topics in the application.

Activity 2.4: Integrate Wikipedia API for Fact Verification

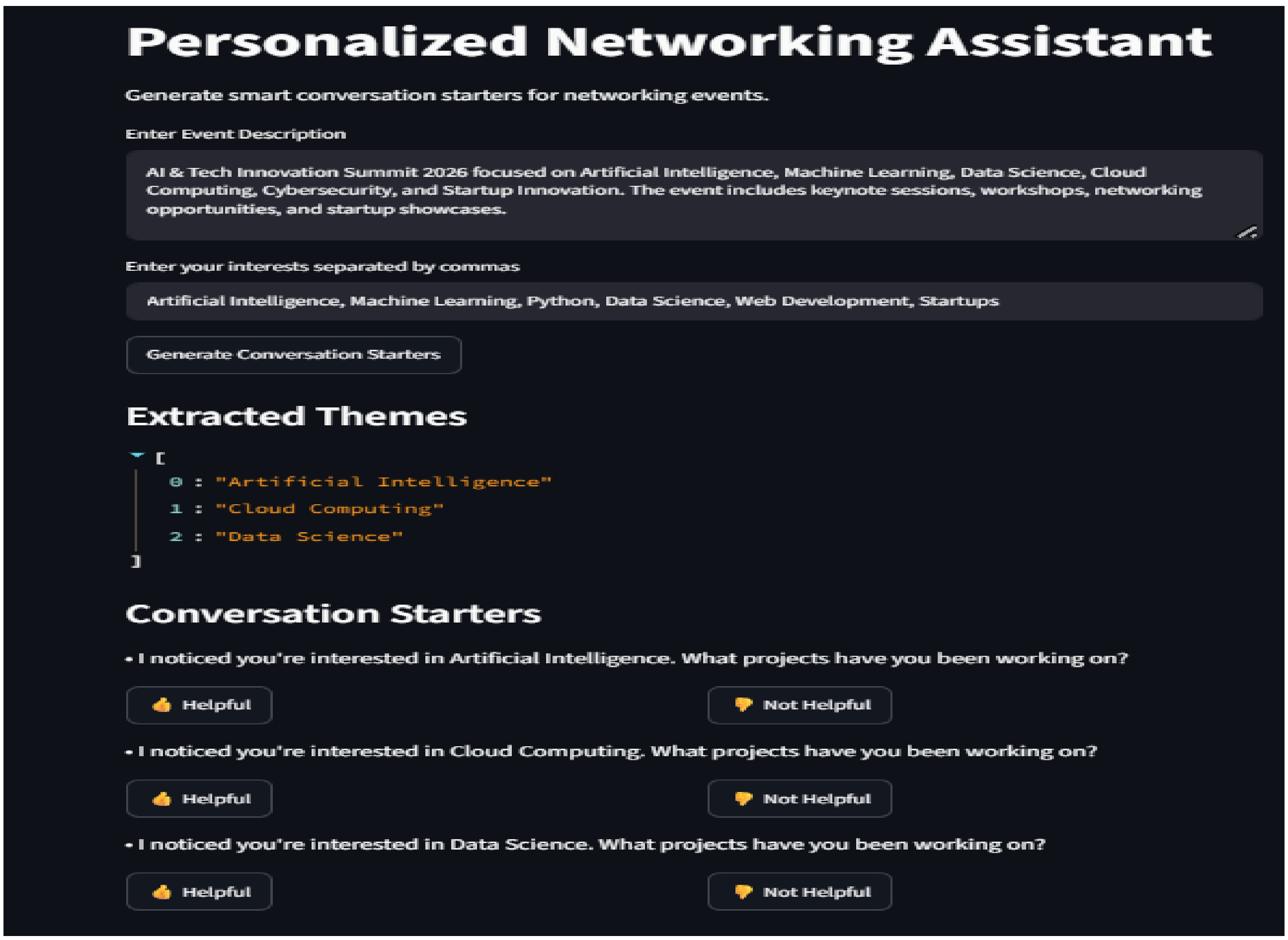
Step 1 – Accept Topic Query: Receive the topic entered by the user for fact checking.
Step 2 – Fetch Wikipedia Summary: Use the Wikipedia API to retrieve reliable information.
Step 3 – Handle Errors: Return a fallback message if no information is found.
Step 4 – Display Verified Result: Show the verified topic summary to the user.

Milestone 3: Frontend Development

In this milestone, we focused on developing the frontend interface for the Personalized Networking Assistant using Streamlit. The frontend allows users to enter profile details, provide event information, generate conversation starters, view verified facts, and access conversation history and feedback pages.

Activity 3.1: Build the Streamlit Web Application

Step 1 – Create Streamlit App File: Build the main frontend file using Streamlit.
Step 2 – Design Page Layout: Add application title, navigation section, input fields, and result display areas.
Step 3 – Connect UI with Backend: Use API calls to communicate with the FastAPI backend.
Step 4 – Run Application Locally: Start the Streamlit app using streamlit run frontend/app.py.



Activity 3.2: Create Profile and Event Input Forms

Step 1 – Add Profile Fields: Allow users to enter their interests and networking context.
Step 2 – Add Event Description Input: Provide a text area for users to describe the networking event.
Step 3 – Validate Required Inputs: Show warning messages if required fields are missing.
Step 4 – Submit Data for Processing: Send user inputs to the backend for AI-based response generation.

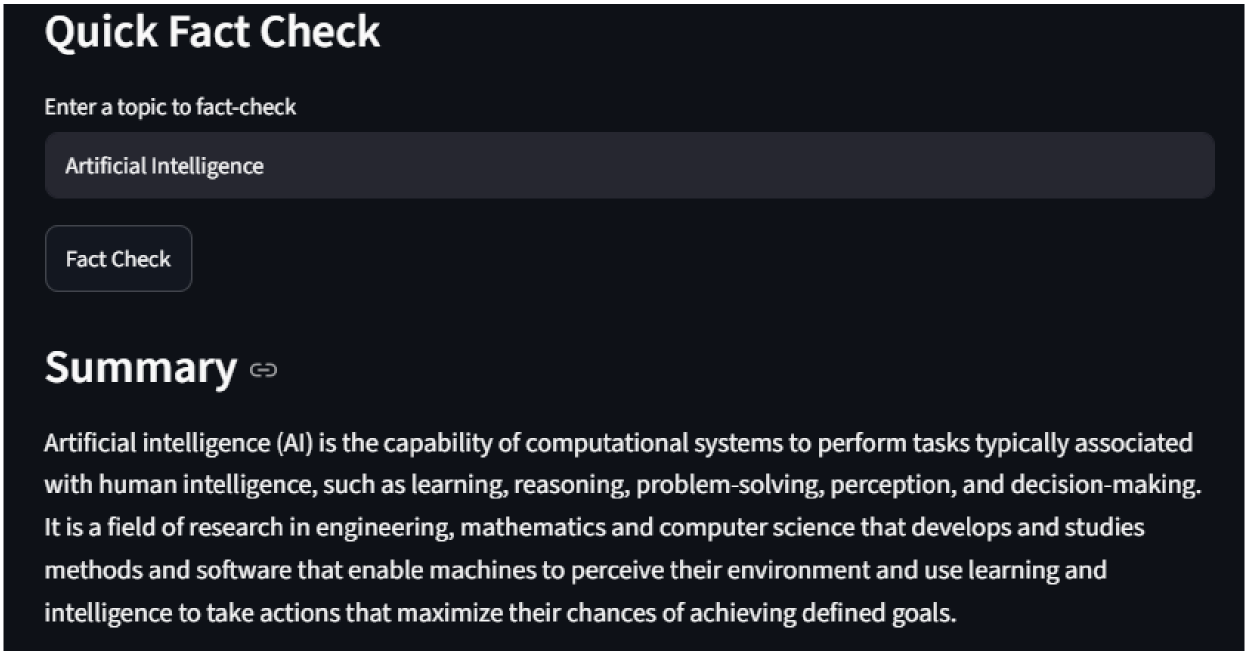
Activity 3.3:Display Conversation Starters, Verified Facts, and AI Suggestions

Step 1 – Show Extracted Topics: Display the networking topics identified from the event description.

Step 2 – Display Conversation Starters: Present AI-generated conversation starters clearly in the UI.

Step 3 – Add Fact-Checking Section: Allow users to verify topics using Wikipedia API results.

Step 4 – Show API Responses: Display generated suggestions and verified information in a readable format.



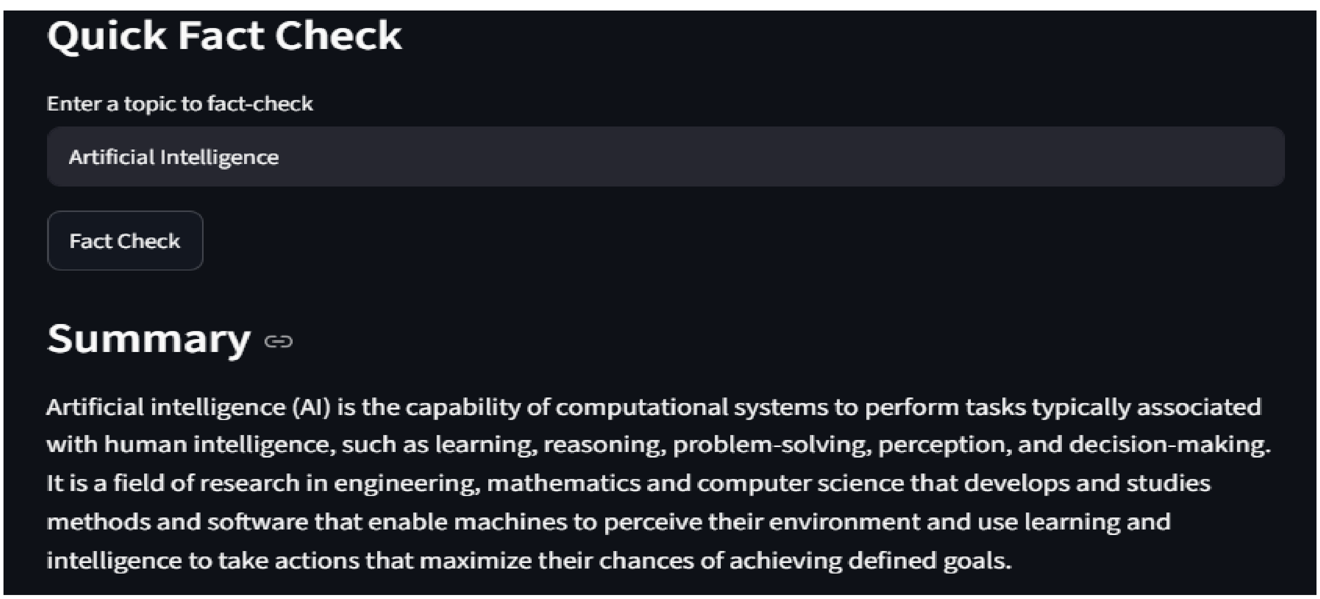
Activity 3.4: Implement Conversation History and Feedback Pages:

Step 1 – Add History View: Show previously generated conversation records.

Step 2 – Add Feedback Options: Allow users to give feedback on generated suggestions.

Step 3 – Display Feedback History: Show stored feedback records for review.

Step 4 – Improve User Experience: Organize sections clearly so users can easily navigate the application.



Milestone 4: Data Management & Integration

In this milestone, we focused on managing user data and integrating the frontend with backend APIs for the Personalized Networking Assistant. This milestone ensures that user inputs, generated conversations, feedback, and history are stored and retrieved properly.

Activity 4.1: Store User Profiles and Interaction History

Step 1 – Capture User Inputs: Collect event descriptions, interests, and networking context from the Streamlit interface.
Step 2 – Store Generated Results: Save extracted topics and generated conversation starters.
Step 3 – Maintain History Records: Store previous user interactions in history.json.
Step 4 – Retrieve Past Conversations: Allow users to view their earlier generated conversation records.

Activity 4.2: Save Feedback and Conversation Logs

Step 1 – Collect User Feedback: Allow users to mark generated suggestions as helpful or not helpful.

Step 2 – Save Feedback Records: Store feedback in feedback.json.

Step 3 – Maintain Conversation Logs: Save conversation starters along with related user interests and topics.

Step 4 – Display Feedback History: Retrieve and show stored feedback records in the frontend.

Activity 4.3: Connect Frontend with Backend APIs Display student transcription.

Step 1 – Configure API Base URL: Connect Streamlit with the FastAPI backend using http://127.0.0.1:8000.
Step 2 – Send User Requests: Use API calls to send event details, interests, and fact-check topics to the backend.
Step 3 – Receive Backend Responses: Display generated conversation starters, extracted topics, fact summaries, history, and feedback.
Step 4 – Verify API Communication: Confirm that all frontend buttons successfully call their respective backend endpoints.

Activity 4.4: Validate AI Responses and Optimize Data Flow

Step 1 – Validate Backend Responses: Check that generated suggestions, topics, and fact-check summaries are returned correctly.

Step 2 – Handle Empty Inputs: Add basic validation to avoid incomplete requests.

Step 3 – Improve Response Flow: Ensure frontend results update smoothly using Streamlit state management.

Step 4 – Optimize Local Storage: Keep history and feedback data organized in JSON files for easy retrieval.

Milestone 5: Testing & Deployment

In this milestone, we focused on validating the functionality, performance, and reliability of the Personalized Networking Assistant. Comprehensive testing was performed on the backend services, APIs, and frontend application before deploying the project locally.

Activity 5.1: Perform Functional and Integration Testing

Step 1 – Test Backend Services: Verify the Event Analyzer, Topic Generator, Fact Checker, History Logger, and Feedback Logger modules.

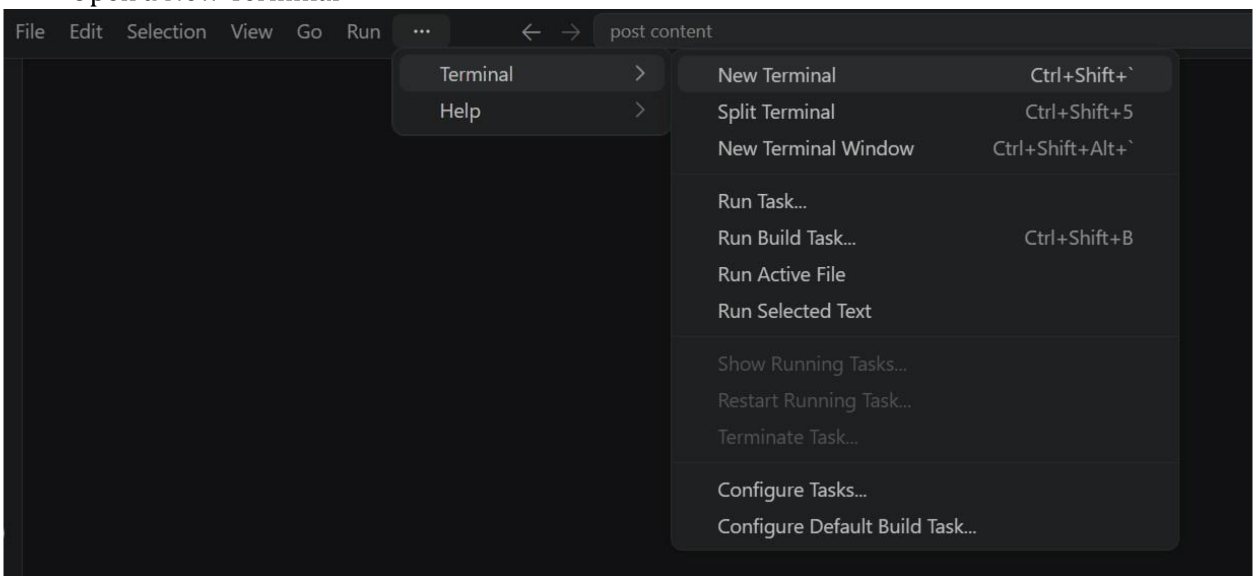
Step 2 – Test FastAPI Endpoints: Validate all API routes using PyTest and FastAPI TestClient.

Step 3 – Perform Integration Testing: Ensure seamless communication between the Streamlit frontend and FastAPI backend.

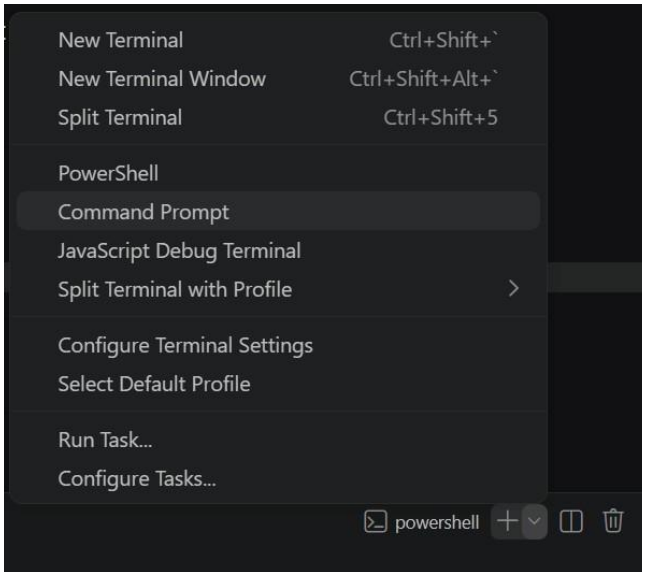
Step 4 – Verify End-to-End Workflow: Test conversation generation, topic classification, fact checking, history, and feedback features.

Set Up a Virtual Environment:

- Open a New Terminal



- Then open “Command Prompt”



- Create and activate a virtual environment, then install all required dependencies from requirements.

```
D:\projects>python -m venv myenv

D:\projects>"d:\projects\venv\Scripts\activate"

(myenv) D:\projects>pip install -r requirements.txt
```

Activity 5.2: Fix Bugs and Optimize Application Performance

- Launch the Streamlit application using the terminal.
- Open the local Streamlit URL in the browser.
- Upload a sample audio file for testing.
- Verify speech transcription, semantic similarity analysis, waveform visualization, scoring, and PDF report generation.
- Confirm that all application modules execute successfully without errors.

```
(myenv) D:\projects>python app.py
```

```
PS C:\Users\dilee\OneDrive\Desktop\Thani\Voice Based Concept Understanding Analyser> python -m streamlit run app.py
2026-06-27 12:11:16.935 Uvicorn server started on 0.0.0.0:8501

  You can now view your Streamlit app in your browser.

  Local URL: http://localhost:8501
  Network URL: http://192.168.31.66:8501
```

Activity 5.3: Deploy the Application Locally and Prepare Project Documentation

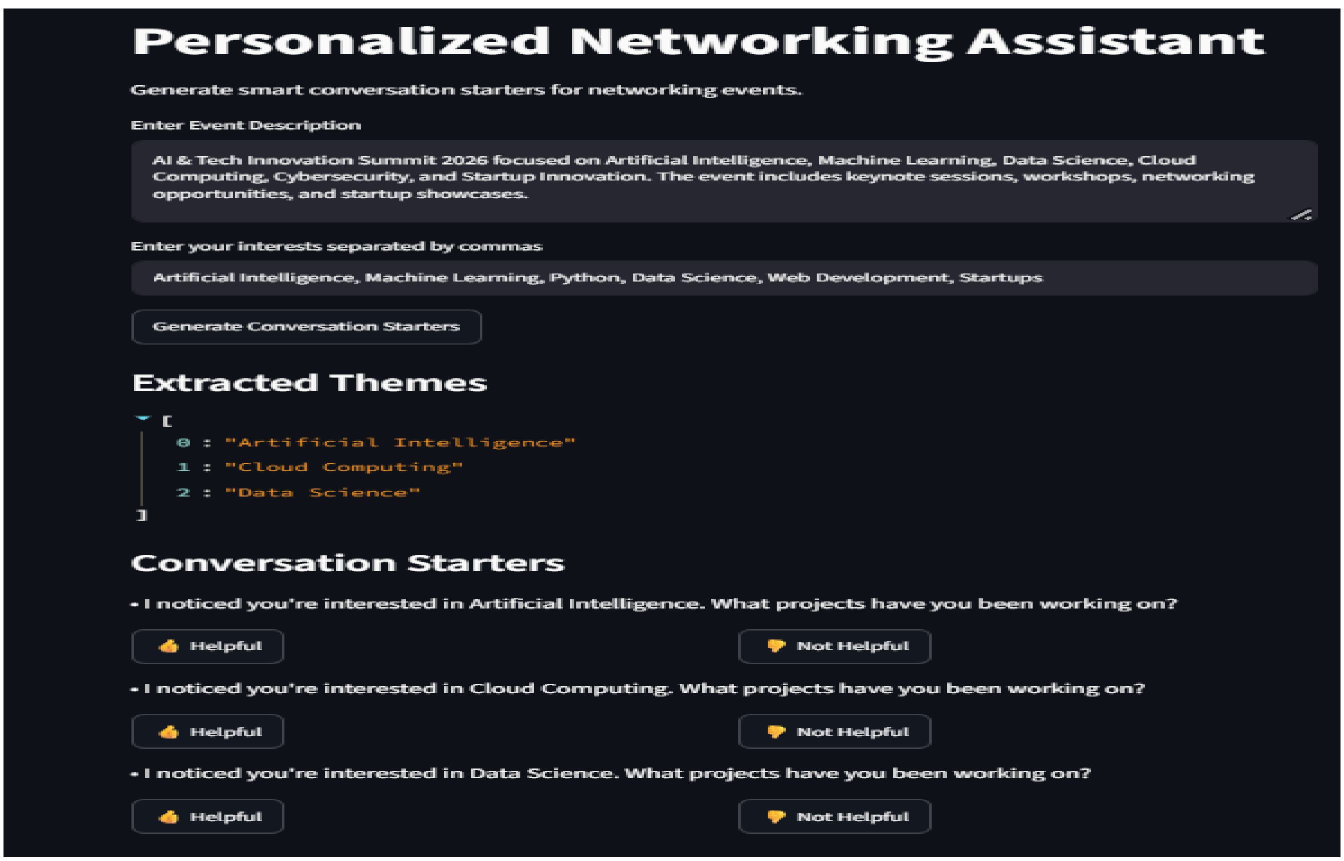
Step 1 – Deploy FastAPI Backend: Launch the backend using Uvicorn with hot reload enabled.

Step 2 – Deploy Streamlit Frontend: Run the Streamlit application and connect it to the backend APIs.

Step 3 – Verify Local Deployment: Confirm the application works correctly through the Streamlit UI and FastAPI Swagger documentation.

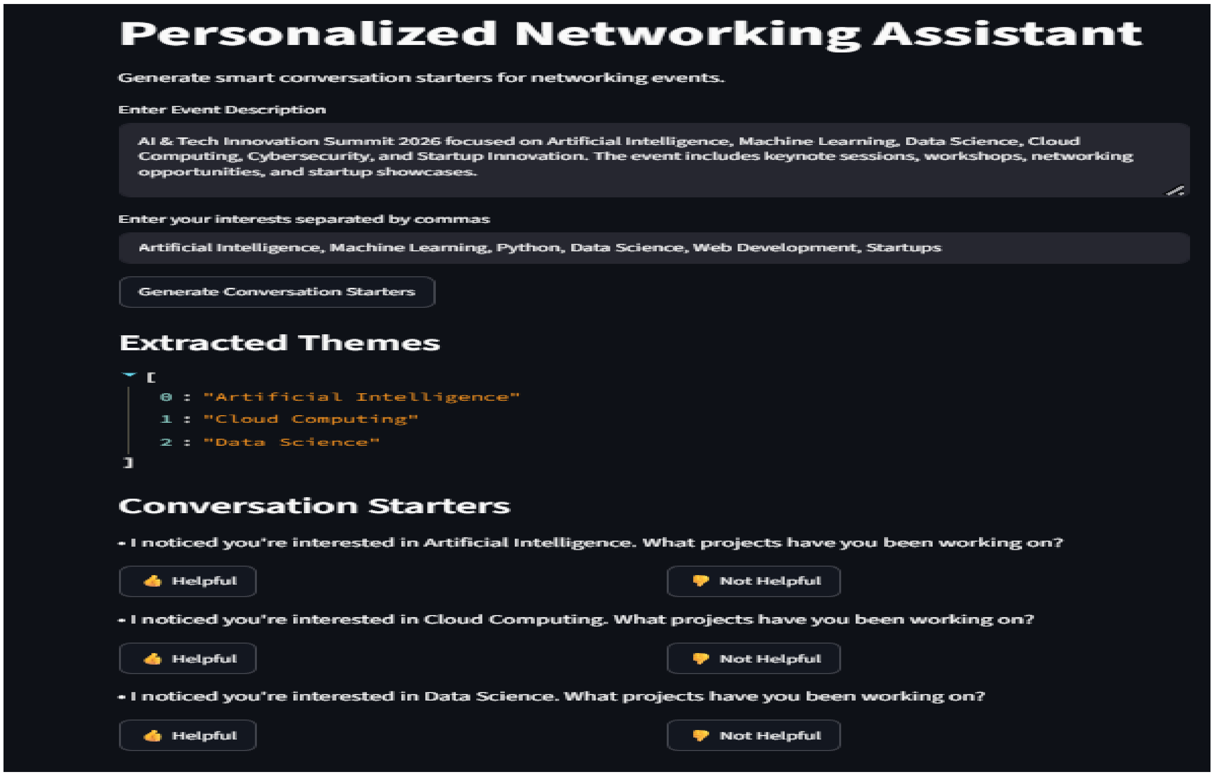
Step 4 – Prepare Documentation: Update the README, project workflow, setup instructions, testing results, and supporting documents for final submission.

```
D:\projects>pip install pyngrok
```



Exploring the Website's Web Pages:

Home / Generator Page:

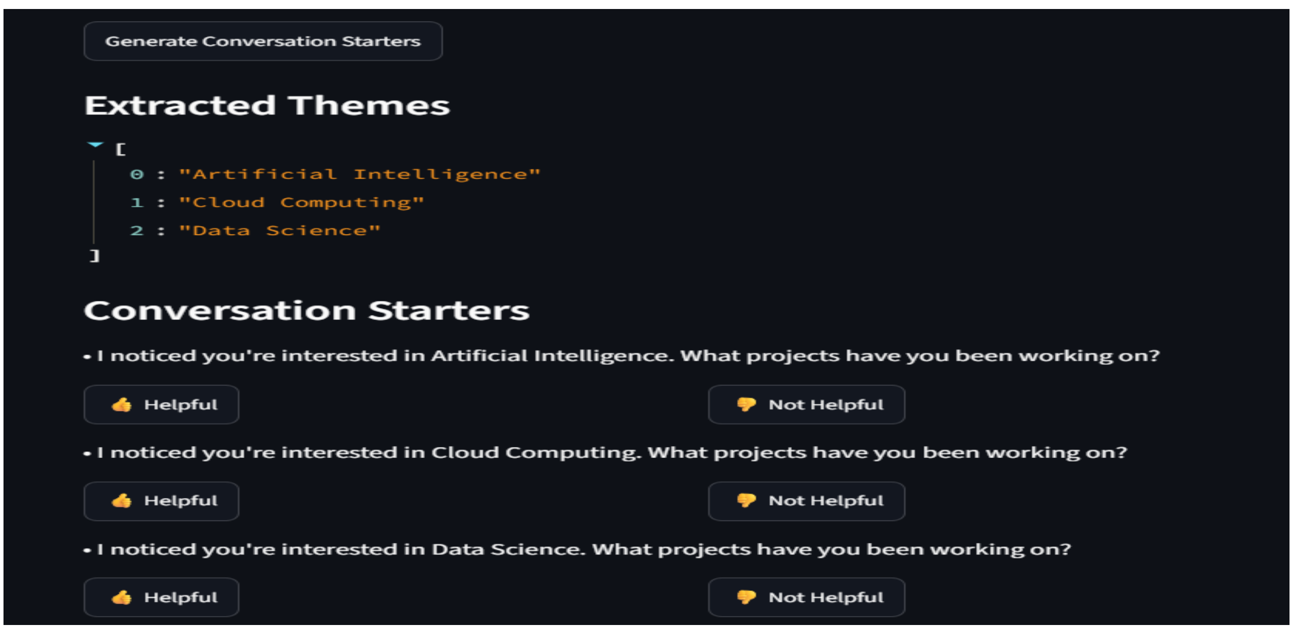


Description: The Home Page serves as the main interface of the Personalized Networking Assistant. Users can enter their profile details, upload or describe an event, and generate AI-powered networking conversation starters. The Streamlit interface provides a simple, responsive, and user-friendly experience.

Input Section:

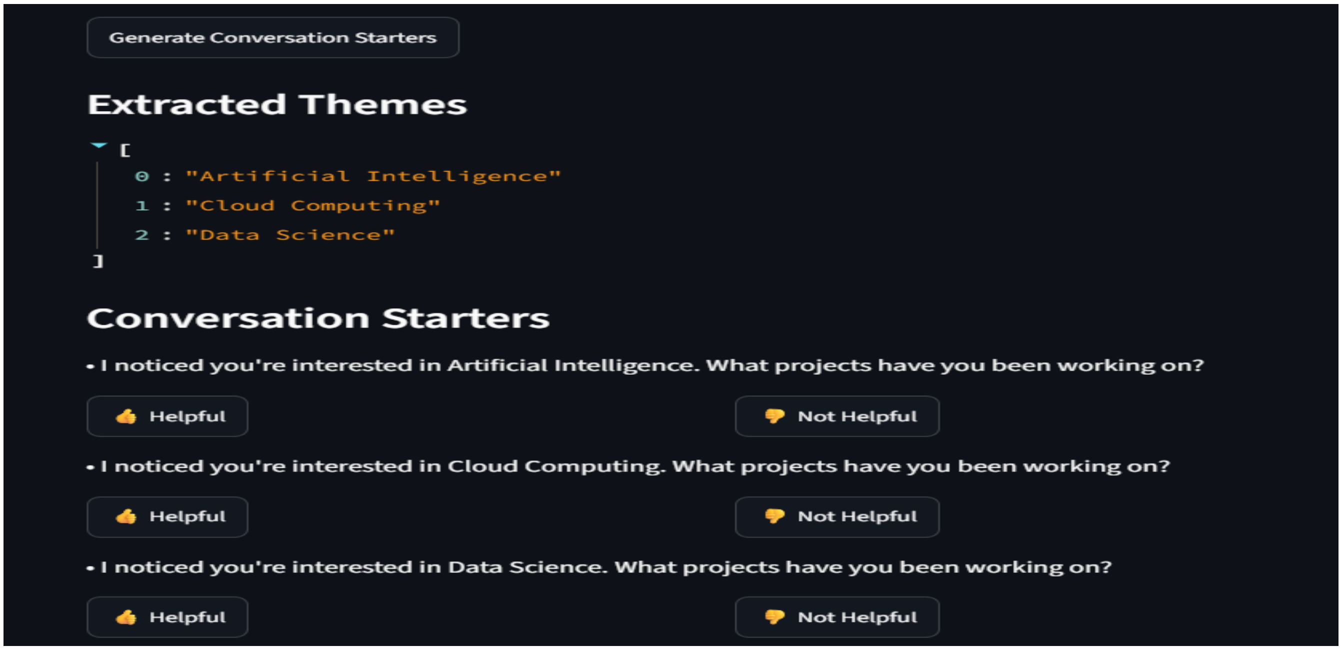
Description: The Input Section allows users to enter their professional profile, event description, networking goals, and conversation preferences. Once all required inputs are provided, users can click the Generate Conversation button to receive personalized networking suggestions.

Results Section:



Description: The Results Section displays AI-generated conversation starters, recommended discussion topics, verified facts retrieved from Wikipedia, and networking tips. These suggestions help users confidently initiate professional conversations and build meaningful connections.

Feedback Section:



Description: The Feedback page enables users to rate AI-generated conversation suggestions and provide comments. The collected feedback is stored for continuous improvement of recommendation quality and user experience.

Conclusion:

The Personalized Networking Assistant successfully integrates GPT-2 Small, DistilBERT, Wikipedia API, FastAPI, and Streamlit into a single AI-powered networking platform. The application generates personalized conversation starters, classifies discussion topics, verifies factual information, maintains conversation history, and collects user feedback. Its modular architecture ensures scalability, maintainability, and an enhanced networking experience for conferences, seminars, workshops, and professional events.